Introduction to NonlinearBSS

In nonlinear blind source separation the observed p-variate data x are assumed to be generated as $x = f(z)$, where f is an unknown injective function and z are independent latent components. The aim is to recover z from x. Without further assumptions this is impossible, but nonstationarity of the components over time or space, or temporal dependence, makes them identifiable.

This vignette illustrates three methods:

- `VAE()`: a variational autoencoder for i.i.d. data (a baseline; not identifiable),
- `iVAE()`: the identifiable variational autoencoder for any data with auxiliary variables,
- `iVAEar()`: iVAE with nonstationary autoregressive latent components.

All plots of the package are made with **ggplot2** and return `ggplot` objects that can be modified further.

The models are fitted with TensorFlow through **keras3**. Install it once with `keras3::install_keras()` and check the setup with `tf_available()`.

Simulated data

`simulate_sources()` creates components whose means and variances change over segments and `mix_data()` mixes them with a random neural network.

```
set.seed(1)
src <- simulate_sources(3, time = 1:3000, n_segments_time = 30)
x <- mix_data(src$sources, n_layers = 3)
```

Auxiliary data

For iVAE the auxiliary variables u should explain how the distribution of the components changes. For a time series, indicators of time segments are a simple choice:

```
A <- aux_matrix(aux_segments(100), x, time = 1:3000)
dim(A)
#> [1] 3000   30
```

Radial basis functions give a smooth and more flexible alternative:

```
A <- aux_matrix(aux_rbf(temporal_basis = c(9, 17, 37)), x, time = 1:3000)
dim(A)
#> [1] 3000   63
```

Fitting iVAE

```
fit <- iVAE(x, aux_rbf(temporal_basis = c(9, 17, 37)), time = 1:3000, epochs = 50,
            verbose = 0, seed = 1)
fit
#> Identifiable variational autoencoder (iVAE)
#>
#> Call: iVAE(x = x, aux = aux_rbf(temporal_basis = c(9, 17, 37)), time = 1:3000,
#>     epochs = 50, seed = 1, verbose = 0)
#> Data: x (3000 observations, 3 variables, format matrix)
```

```
#> Latent components: 3
#> Auxiliary data: radial basis functions, spatial = 2, 9, temporal = 9, 17, 37 (gaussian) [63 variable
#> Distributions: source gaussian, error gaussian (sigma 0.02)
#> ELBO: -3.1172
absolute_mean_correlation(fit$S, src$sources)
#> [1] 0.9565594
```

For comparison, the plain `VAE()` has no auxiliary variables. It is a baseline whose components are not identifiable in general, so its recovery is not guaranteed:

```
fit_vae <- VAE(x, latent_dim = 3, epochs = 50, verbose = 0, seed = 1)
absolute_mean_correlation(fit_vae$S, src$sources)
#> [1] 0.7132899
```

The mean correlation coefficient compares the estimates with the true components up to permutation and sign; 1 means perfect recovery. The components are stored as `fit$S` and are returned in the format of the data by `components()`.

```
head(components(fit))
#>             IC1        IC2        IC3
#> [1,]  2.0283578  0.1393474 -0.4877269
#> [2,] -1.6412711 -0.6063424 -1.6819368
#> [3,]  1.0652925 -0.6113633 -1.2268908
#> [4,] -0.7140939  0.9349620 -0.6195569
#> [5,]  0.4349818 -1.6999949 -1.6443090
#> [6,] -0.8067482  0.4258073 -0.2712600
```

Choosing the number of latent components

In practice the number of latent components is unknown. Following Sipilä et al. (2025, *Neural Networks*; 2026, *Spatial Statistics*), it can be selected by fitting models with different numbers of components L and maximising the profile AIC

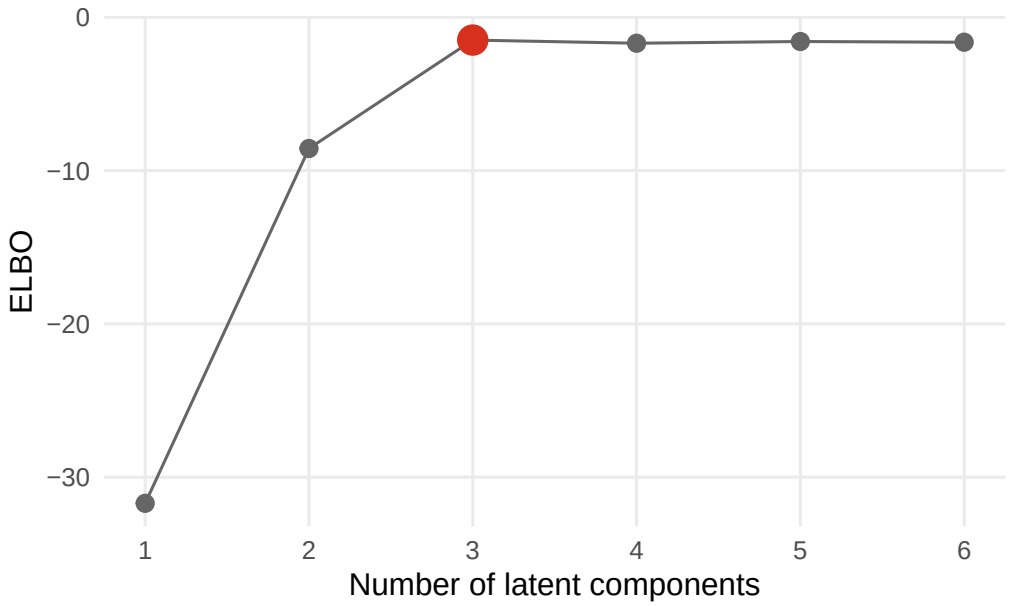
$$\text{pAIC} = \text{ELBO} - L,$$

where the ELBO is the average evidence lower bound per observation. Here the three sources are mixed into six observed variables, and models with one to six components are compared:

```
x6 <- mix_data(src$sources, n_layers = 3, output_dim = 6)
sel <- select_latent_dim(x6, latent_dims = 1:6, aux = aux_segments(100),
                         time = 1:3000, epochs = 50, verbose = 0, seed = 1)
sel
#> Selection of the number of latent components
#>
#>  latent_dim    elbo    pAIC selected
#>           1 -31.703 -32.703
#>           2  -8.562 -10.562
#>           3  -1.491  -4.491        *
#>           4  -1.696  -5.696
#>           5  -1.585  -6.585
#>           6  -1.631  -7.631
#>
#> Selected by the largest pAIC = ELBO - number of components: 3
```

The ELBO increases steeply until the true number of components and stays nearly constant after that:

```
plot(sel)
```



The selected model is kept in the result. Models that were fitted separately, for example in different sessions, can be compared with `latent_dim_selection(list(fit1, fit2, ...))`. The models should differ only in the number of components, since the ELBO also depends on the data, the auxiliary variables and `error_dist_sigma`.

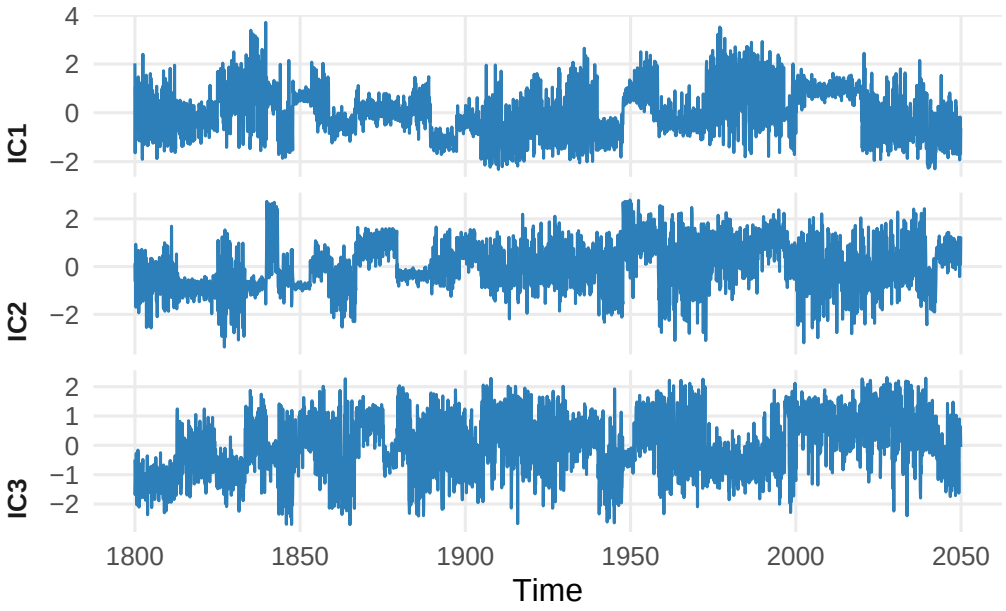
Time series formats

Time series objects (`ts`, `xts`, `zoo`, `tsibble`) can be used directly, and the results are returned in the same format.

The time points of these objects are measured in observation steps (here months), as for matrices, so the auxiliary variables are specified in the same way for all data formats:

```
xt <- ts(x, start = c(1800, 1), frequency = 12)
fit_ts <- iVAE(xt, aux_rbf(temporal_basis = c(9, 17, 37)), epochs = 50, verbose = 0, seed = 1)
class(components(fit_ts))
#> [1] "mts"    "ts"     "matrix" "array"
absolute_mean_correlation(fit_ts$S, src$sources)
#> [1] 0.9565594
```

```
plot(fit_ts)
```



A second simulated example: spatio-temporal data

`simulate_sources()` also creates spatio-temporal components, with nonstationarity over space and time and, optionally, autoregressive dependence. Here three latent components are observed at 60 random locations and at 40 time points, and the three components are mixed into five observed variables with a two-layer random mixing function:

```
set.seed(1)
n_s <- 60
n_t <- 100
xy <- cbind(lon = runif(n_s), lat = runif(n_s))
coords <- xy[rep(seq_len(n_s), n_t), ]
time <- rep(seq_len(n_t), each = n_s)
src_st <- simulate_sources(3, coords = coords, time = time,
                           n_segments_space = 6, n_segments_time = 5,
                           ar_range = c(0.3, 0.9))
x_st <- mix_data(src_st$sources, n_layers = 2, output_dim = 5)
dat <- data.frame(x_st, coords, time = time)
vars_st <- colnames(x_st)
```

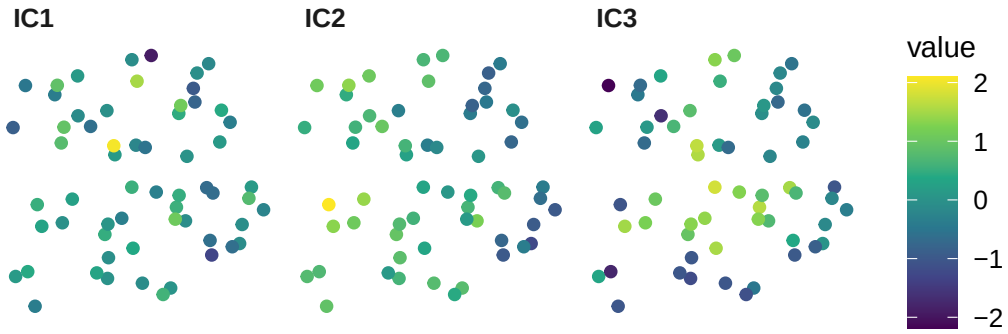
The data frame is used with the names of the coordinate, time and variable columns. The radial basis functions of `aux_rbf()` are the auxiliary variables. `VAE()` ignores the coordinates and times, `iVAE()` treats the observations as independent given the auxiliary variables, and `iVAEar()` also models the autoregressive dependence:

```
aux <- aux_rbf(spatial_basis = c(2, 9), temporal_basis = c(9, 17))
fit_st_vae <- VAE(dat[vars_st], latent_dim = 3, epochs = 60, verbose = 0, seed = 1)
fit_st_ivae <- iVAE(dat, aux, coords = c("lon", "lat"), time = "time",
                    vars = vars_st, latent_dim = 3, epochs = 60, verbose = 0,
                    seed = 1)
fit_st_ar <- iVAEar(dat, aux, coords = c("lon", "lat"), time = "time",
                    vars = vars_st, latent_dim = 3, epochs = 60, verbose = 0,
                    seed = 1)
sapply(list(VAE = fit_st_vae, iVAE = fit_st_ivae, iVAEar = fit_st_ar),
       function(f) absolute_mean_correlation(f$S, src_st$sources))
```

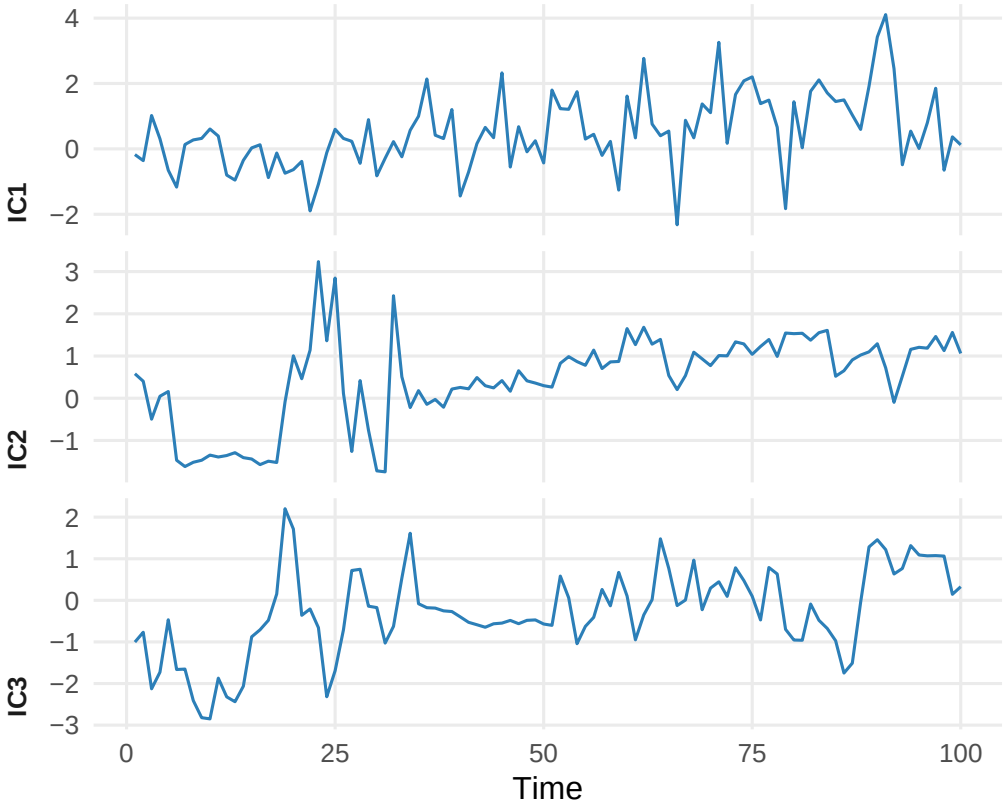
```
#>       VAE      iVAE    iVAEar
#> 0.8760115 0.9822837 0.9917308
```

The estimated components as a map at the last time point, and as a time series at the first location:

```
plot(fit_st_ar)
```




```
plot(fit_st_ar, type = "series", series = 1)
```



Since the result is a `ggplot` object, it can be customised with the usual ggplot2 functions, for example `plot(fit_st_ar) + ggplot2::ggtitle("iVAEar")`.

Spatio-temporal data

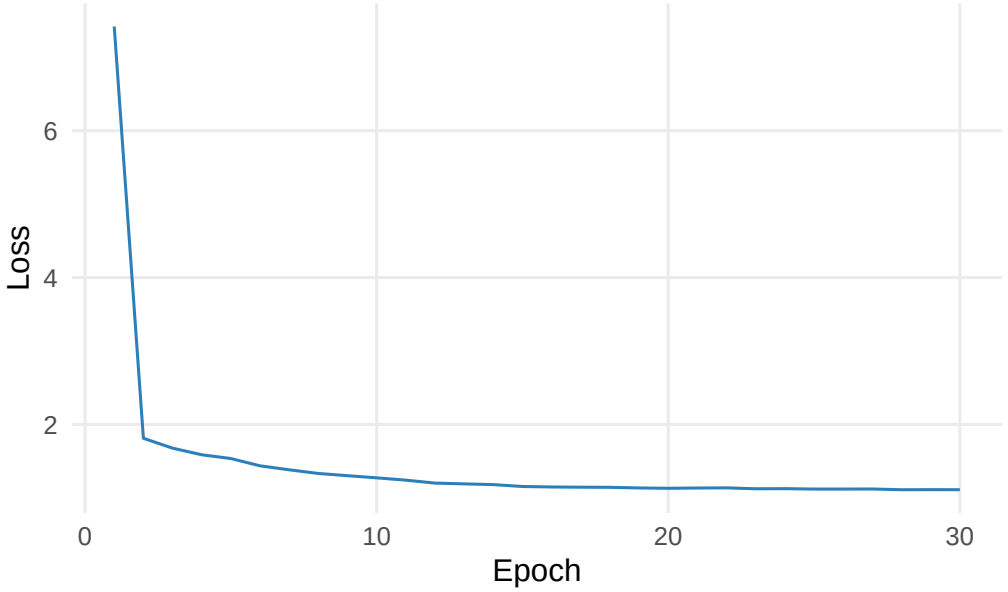
The data set `pollution_northern_italy` contains daily air quality measurements at 50 stations. Here, the observations from January 2020 to April 2022 are used for training, May 2022 as forecasting test data. Five stations are set aside to test spatial interpolation. The data frame is used with the column names of the coordinates, time points and variables.

```r
train_subset_time <- subset(pollution_northern_italy, time < as.Date("2022-05-01"))
stations <- unique(train_subset_time$AirQualityStation)
set.seed(2)
test_stations <- sample(stations, 5)
train_subset <- subset(train_subset_time, !AirQualityStation %in% test_stations)
vars <- c("mean_O3", "mean_NO2", "mean_PM10")
```

`iVAEar()` models the components as nonstationary autoregressive processes. The model is trained for 30 epochs with RBF auxiliary variables. The temporal RBFs are formed based on the seasonal time point (1 to 365) instead of the absolute time point in order to allow the model to generalize better to future.

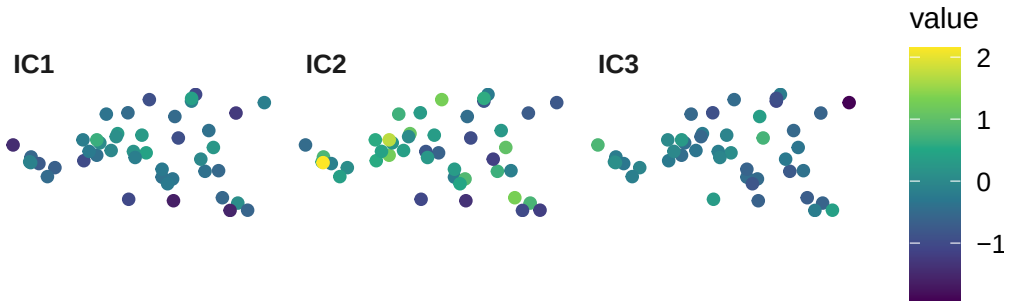
```r
fit <- iVAEar(train_subset, aux = aux_rbf(spatial_basis = c(2, 9),
                                          temporal_basis = c(9, 17), seasonal_period = 365),
              aux_hidden_units = c(16),
              coords = c("Longitude", "Latitude"), time = "time", vars = vars,
              epochs = 30, verbose = 0, seed = 1)
summary(fit)
#> Identifiable variational autoencoder with autoregressive components (iVAEar)
#>
#> Call: iVAEar(x = train_subset, aux = aux_rbf(spatial_basis = c(2, 9),
#>     temporal_basis = c(9, 17), seasonal_period = 365), coords = c("Longitude",
#>     "Latitude"), time = "time", vars = vars, aux_hidden_units = c(16),
#>     epochs = 30, seed = 1, verbose = 0)
#>
#> Data: train_subset
#>   observations: 38295
#>   variables: 3
#>   format: data.frame
#> Latent components: 3
#> Autoregressive order: 1
#> Auxiliary data: radial basis functions, spatial = 2, 9, temporal = 9, 17 (gaussian) + seasonal perio
#>
#> Network: hidden units 128-128-128, auxiliary function 16, activation leaky_relu
#> Distributions: source gaussian, error gaussian
#>
#> Training: 30 epochs, final loss 1.11
#> ELBO: -1.0568
#> Largest absolute correlation between components: 0.494
#>
#> Prior variance of the standardised components over the data:
#>         0%   50%  100%
#> IC1 0.0522 0.150 0.485
#> IC2 0.1558 0.310 0.713
#> IC3 0.0585 0.168 0.559
#>
#> Autoregressive coefficients over the data:
#>  component lag   min  mean   max
#>        IC1   1 0.313 0.690 0.936
#>        IC2   1 0.410 0.742 0.927
#>        IC3   1 0.551 0.755 0.928
```

```r
plot(fit, type = "history")
```

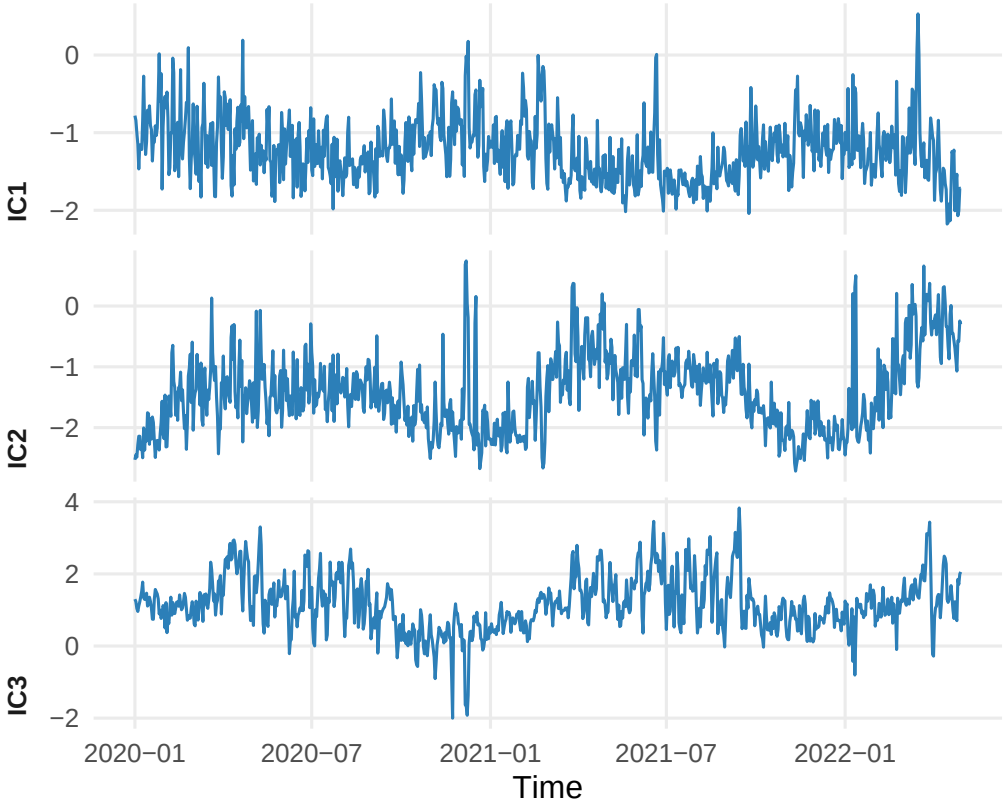


The components can be shown as maps at a given day, or as time series at a station:

```
plot(fit, time = as.Date("2022-04-01"))
```




```
plot(fit, type = "series", series = 1)
```



Forecasting and interpolation

The trend, variance and autoregressive coefficient functions can be evaluated anywhere, and the components can be forecast from their last values by iterating the autoregressive prior. With `space = "data"` the predicted components are decoded to the observed variables. Here we forecast the first week of May 2022, one to seven days after the end of the training data:

```
test_time <- subset(pollution_northern_italy,
                    time < as.Date("2022-05-08") & time > as.Date("2022-04-30"))
pred_time <- predict(fit, test_time, type = "forecast",
              coords = c("Longitude", "Latitude"), time = "time",
              space = "data")
head(pred_time)
#>       Longitude Latitude       time   mean_O3 mean_NO2 mean_PM10
#> 63663  11.43317 46.58967 2022-05-01 106.77232 5.209356  11.31909
#> 63664  11.43317 46.58967 2022-05-02 101.98539 5.529307  11.70152
#> 63665  11.43317 46.58967 2022-05-03  97.82937 5.778080  11.92008
#> 63666  11.43317 46.58967 2022-05-04  94.29707 5.888305  12.04490
#> 63667  11.43317 46.58967 2022-05-05  91.27018 6.015951  12.07333
#> 63668  11.43317 46.58967 2022-05-06  88.76884 6.126650  12.03370
```

The mean absolute error of the ozone forecasts by forecast day, compared with the standard deviation of ozone in the test period:

```
round(c(MAE = mean(abs(pred_time$mean_O3 - test_time$mean_O3)),
        sd = sd(test_time$mean_O3)), 1)
#>  MAE   sd
#> 15.3 18.8
```

Spatial interpolation uses the trend function at the locations of the five stations not used in fitting:

```
test_space <- subset(train_subset_time, AirQualityStation %in% test_stations)
pred_space <- predict(fit, test_space, type = "trend",
                  coords = c("Longitude", "Latitude"), time = "time",
                  space = "data")
round(c(MAE = mean(abs(pred_space$mean_O3 - test_space$mean_O3)),
        sd = sd(test_space$mean_O3)), 1)
#>  MAE   sd
#> 17.8 33.2
```

The predictions can be made also to future time points of new stations by using the trend function:

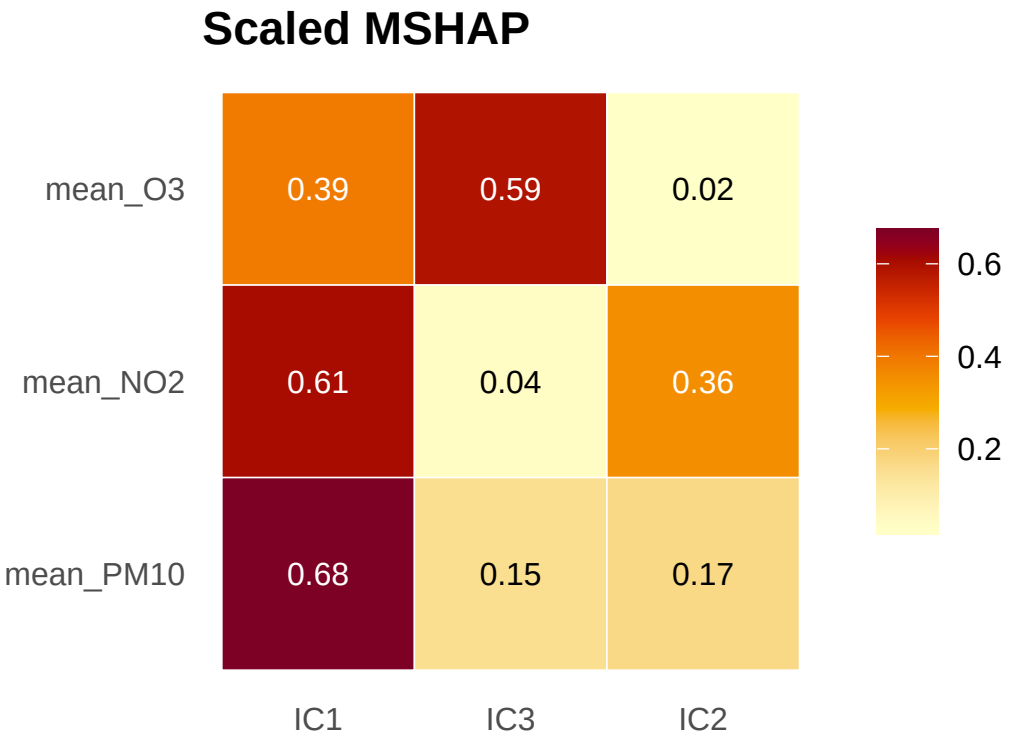
```
test_space_time <- subset(pollution_northern_italy, AirQualityStation %in% test_stations &
                          time < as.Date("2022-05-08") & time > as.Date("2022-04-30"))
pred_space_time <- predict(fit, test_space_time, type = "trend",
                  coords = c("Longitude", "Latitude"), time = "time",
                  space = "data")
round(c(MAE = mean(abs(pred_space_time$mean_O3 - test_space_time$mean_O3)),
        sd = sd(test_space_time$mean_O3)), 1)
#>  MAE   sd
#> 20.3 24.3
```

Interpretation

`mshap()` measures how much each latent component contributes to each observed variable by mean absolute kernel SHAP values of the decoder. The scaled values sum to one for each variable. Sipilä et al. (2026, *Spatial Statistics*) used them to interpret the latent components of weather and air pollution data over Northern Italy.

```
ms <- mshap(fit, n_sample = 500, n_background = 50, seed = 1)
ms
#> Scaled mean absolute SHAP values (rows: variables, columns: components)
#>
#>              IC1   IC3   IC2
#> mean_O3   0.393 0.590 0.017
#> mean_NO2  0.605 0.040 0.355
#> mean_PM10 0.676 0.152 0.172
#> Average   0.558 0.261 0.181
#>
#> Based on 500 observations and 50 background observations.
```

```
plot(ms)
```



Spatial data formats

With an `sf` or `sftime` object the coordinates and time points are taken from the object, and the results are again `sf` objects. For points, the coordinates are those of the geometry:

```
s <- sf::st_as_sf(dat, coords = c("lon", "lat"))
fit_sf <- iVAEar(s, aux, time = "time", vars = vars_st, latent_dim = 3,
                 epochs = 60, verbose = 0, seed = 1)
head(components(fit_sf), 3)
#> Simple feature collection with 3 features and 4 fields
#> Geometry type: POINT
#> Dimension:     XY
#> Bounding box:  xmin: 0.2655087 ymin: 0.2936034 xmax: 0.5728534 ymax: 0.9128759
#> CRS:           NA
#>   time        IC1       IC2        IC3                    geometry
#> 1    1 -0.1744864 0.5799103 -1.0028925 POINT (0.2655087 0.9128759)
#> 2    1  1.0015271 0.3588701 -0.1335403 POINT (0.3721239 0.2936034)
#> 3    1  0.5771135 0.6203762 -1.1136303 POINT (0.5728534 0.4590657)
```

Spatio-temporal objects of the **sp** and **spacetime** packages (`STFDF`) are also accepted, and the results are returned as `STFDF` objects:

```
sp_pts <- sp::SpatialPoints(xy)
st <- spacetime::STFDF(sp_pts, as.POSIXct("2020-01-01", tz = "UTC") +
                       86400 * (seq_len(n_t) - 1),
                       data.frame(x_st))
fit_stfdf <- iVAEar(st, aux, latent_dim = 3, epochs = 20, verbose = 0, seed = 1)
class(components(fit_stfdf))
```

Large data

For large spatio-temporal data the auxiliary matrix can become very large. With `batch_aux = TRUE` the auxiliary variables are computed for each batch inside the network from the coordinates and time points, which gives the same model with much less memory:

```
fit <- iVAEar(pollution_northern_italy, aux_rbf(),
              coords = c("Longitude", "Latitude"), time = "time", vars = vars,
              batch_aux = TRUE)
```

Saving models

The trained weights are stored in the fitted object, so models can be saved with `save_nlbss()` (or `saveRDS()`) and used in a new session:

```
file <- tempfile(fileext = ".rds")
save_nlbss(fit, file)
fit2 <- load_nlbss(file)
all.equal(predict(fit2, type = "trend"), predict(fit, type = "trend"))
#> [1] TRUE
```

References

- Hyvärinen, A. and Morioka, H. (2016). Unsupervised feature extraction by time-contrastive learning and nonlinear ICA. *Advances in Neural Information Processing Systems*, 29.
- Khemakhem, I., Kingma, D., Monti, R. and Hyvärinen, A. (2020). Variational autoencoders and nonlinear ICA: A unifying framework. *AISTATS 2020*, 2207–2217.
- Sipilä, M., Nordhausen, K. and Taskinen, S. (2024). Nonlinear blind source separation exploiting spatial nonstationarity. *Information Sciences*, 665, 120365. https://doi.org/10.1016/j.ins.2024.120365
- Sipilä, M., Cappello, C., De Iaco, S., Nordhausen, K. and Taskinen, S. (2025). Modelling multivariate spatio-temporal data with identifiable variational autoencoders. *Neural Networks*, 181, 106774. https://doi.org/10.1016/j.neunet.2024.106774
- Sipilä, M., Nordhausen, K. and Taskinen, S. (2025). Identifiable autoregressive variational autoencoders for nonlinear and nonstationary spatio-temporal blind source separation. *ECML PKDD 2025*, Lecture Notes in Computer Science 16019, 362–380. https://doi.org/10.1007/978-3-032-06109-6_21
- Sipilä, M., De Iaco, S., Cappello, C., Nordhausen, K., Palma, M. and Taskinen, S. (2026). Enhancing identifiable variational autoencoder in the presence of missing values and auxiliary covariates for spatio-temporal ozone modeling. *Environmetrics*, 37(6), e70121. https://doi.org/10.1002/env.70121
- Sipilä, M., Cappello, C., De Iaco, S., Nordhausen, K., Palma, M. and Taskinen, S. (2026). Identifiable variational autoencoders for interpretable dimension reduction and prediction in multivariate spatio-temporal processes. *Spatial Statistics*, 101054. https://doi.org/10.1016/j.spasta.2026.101054